

1주차 파머완_백재은(ML 세션)

Topic / 파이썬 기반의 머신러닝과 생태계 이해

1. 머신러닝의 개념

개념

general → 애플리케이션 수정 X 데이터 기반 패턴 학습 • 결과 예측 알고리즘 기법

Problem → 프로그램 로직으로 조건, 환경 변수, 규칙 반영 = 고비용, 저효율(정확성 향상 X)

Solution → 데이터 기반 패턴 분석 및 예측 = Predictive Analysis

분류

- Supervised Learning
 - Classification
 - Regression
- Un-Supervised Learning
 - Reinforcement Learning

머신러닝과 데이터 품질의 연관성

Law of "Garbage in, Garbage Out"

→ 데이터 이해, 가공, 처리, 추출 후 최적화의 중요성

2. 주요 패키지

- 머신러닝 패키지 - Scikit-Learn
- 행렬/선형대수/통계 패키지 - NumPy, SciPy

- 데이터 핸들링 - Pandas
- 시각화 - Matplotlib, Seaborn
- Third-Party Library
- IPython - Jupyter Notebook

3. 넘파이 (Numerical Python)

- 선형대수 기반의 프로그램 지원 패키지
- 저수준 언어(C/C++) 기반 호환 API, 제공 → 데이터 송신 및 API 호출 통합 기능 제공
 ↳ 파이썬 언어의 수행 성능 제약 → 중요 파트 C/C++ 작성 후 API 호출 ex. 텐서플로

a. 넘파이 ndarray

ndarray - 넘파이 기반 데이터 타입

넘파이 array() 함수 - 인자 input, ndarray 변환 output

ndarray 배열의 shape 변수 - ndarray 크기(행렬의 수) in 튜플 형태, 배열의 차원

array() 함수 인자 - 파이썬 리스트 객체 사용(배열 차원, 크기 표현 easy)

b. ndarray 데이터 타입

데이터값: 숫자 값(int형, unsigned int형, float형, complex 타입), 문자열 값, bool 값 모두 가능

데이터 타입: 연산 특성상 같은 데이터 타입만 가능 → dtype 속성으로 확인 가능

ndarray 내 데이터값의 타입 변경: astype() 메서드, 인자 - 원하는 타입을 문자열로 지정

c. ndarray 편리하게 생성하기 / arange(), zeros(), ones()

When? 특정 크기와 차원을 가진 ndarray → 연속값, 0, 1로 초기화하여 쉽게 생성해야 하는 필요가 있는 경우 / ex. 테스트용, 대규모 데이터 일괄 초기화

arange() ⇒ 0 ~ 함수 인자값 -1 까지의 값을 순차적으로 ndarray의 데이터값으로 변환

default 함수 인자 = stop 값

start 값 부여 시 0이 아닌 다른 값부터 시작한 연속 값 부여 가능

zeros() ⇒ 함수 인자로 튜플 형태의 shape 값 입력 시 모든 값을 0으로 채운 해당 shape을 가진 ndarray 반환

ones() ⇒ 함수 인자로 튜플 형태의 shape 값 입력 시 모든 값을 1로 채운 해당 shape을 가진 ndarray 반환

*함수 인자로 dtype 정하지 않을 시 default로 float64 형의 데이터로 ndarray 채움

d. reshape() - ndarray 차원, 크기 변경

reshape() 메서드: ndarray를 특정 차원 및 크기로 변환

함수 인자: 변환을 원하는 크기

*지정된 사이즈로 변경 불가 시, 오류 발생

e. Indexing - 넘파이 ndarray 데이터 세트 선택

- 특정 데이터만 추출: 원하는 위치의 인덱스 값 지정 - 해당 위치의 데이터 반환

*다차원 ndarray의 경우, 축(axis)를 가짐 / 2차원 axis 0 = row, axis 1 = col, 3차원 axis 2 = height

- 슬라이싱: 연속된 인덱스 상의 ndarray 추출 - ':' 기호 사이 시작 인덱스, 종료 인덱스 표시 시 시작 인덱스 ~ 종료 -1 인덱스 위치 데이터의 ndarray 반환

*시작, 종료 인덱스 생략 가능/ 생략 시 시작 = 처음 인덱스 0, 종료 = 맨 마지막 인덱스

- 팬시 인덱싱: 일정한 인덱싱 집합을 리스트, ndarray 형태 지정, 해당 위치 데이터 ndarray 반환

- 불린 인덱싱: 특정 조건에 해당하는지 여부 (T/F) 값 인덱싱 집합을 기반으로 True 해당 인덱스 위치 데이터 ndarray 반환

동작 단계: ndarray의 필터링 조건 [] 안에 기재 → F 값 무시, T 값 해당 인덱스 값만 저장 → 저장된 인덱스 데이터 세트에 ndarray 조회

f. sort(), argsort() - 행렬의 정렬

- 행렬 정렬

→ np.sort() - 넘파이에서 sort() 호출 / 원 행렬 유지, 원 행렬의 정렬된 행렬 반환

→ ndarray.sort() - 행렬 자체에서 sort() 호출 / 원 행렬 자체를 정렬한 형태로 변환, 반환 값 = None

- 정렬된 행렬의 인덱스를 반환하기

→ np.argsort() - 원본 행렬 정렬, 기존 원본 행렬의 원소에 대한 인덱스를 필요로 할 때 / 정렬 행렬의 원본 행렬 인덱스를 ndarray 형태로 반환

g. 선형대수 연산 - 행렬 내적과 전치 행렬 구하기

- 행렬 내적(행렬 곱) → np.dot(A, B) - A행렬과 B행렬의 내적 연

- 전치 행렬 → np.transpose() - 원 행렬에서 행과 열 위치를 교환한 원소로 구성한 행렬

4. 판다스 - 데이터 핸들링

Intro

기능

→ 행과 열로 이뤄진 2차원 데이터를 효율적으로 가공/처리

→ 다양한 유형의 분리 문자로 칼럼을 분리한 파일을 쉽게 DataFrame으로 로딩

핵심 객체

→ DataFrame, 여러 개의 행과 열로 이뤄진 2차원 데이터를 담는 데이터 구조체 (칼럼 여러 개)

중요 객체

→ Index: 개별 데이터를 고유하게 식별하는 Key 값

→ Series: 칼럼이 하나인 데이터 구조체

Pandas 시작

a. Titanic_train.csv 파일 분석 예제 (Ex 7)를 통한 Pandas API

다양한 포맷으로 된 파일을 DataFrame으로 로딩할 수 있는 판다스의 API

- read_csv() → CSV 파일 포맷 변환, 디폴트 필드 구분 문자 = 콤마 / 콤마가 아닌 구분 문자 기반의 파일 포맷의 경우, 인자 sep에 해당 구분 문자 입력 시 DataFrame으로 변환 가능
- read_table() → 디폴트 필드 구분 문자 = 탭('\t') 문자
- read_fwf() → Fixed Width, 고정 길이 기반의 칼럼 포맷 변환

b. DataFrame 기본

- read_csv(filepath_or_buffer, sep=',, ...) ⇒ filepath 가능 중요, 나머지 미지정 시 디폴트 값 할당
- Filepath = 로드하려는 데이터 파일의 경로 포함 파일명
- DataFrame.head() → 맨 앞 n개의 로우 반환
- shape → DataFrame의 행과 열을 튜플 형태로 반환 / 행렬 크기 알아보기 Good
- 메타 데이터 조회 메소드
 - info(): 총 데이터 건수, 데이터 타입, Null 건수
 - describe(): 칼럼별 숫자형 데이터 값의 분포도, object 타입 칼럼 출력 제외
count: Not Null 건수, mean: 전체 데이터 평균값, std: 표준편차, min: 최솟값, max: 최댓값
 - 25/50/75%: 각각의 percentile 값
 - 카테고리 칼럼: 특종 범주에 속하는 값을 코드화한 칼럼
 - value_counts(): 지정된 칼럼의 데이터값 건수 반환

c. 넘파이 ndarray, 리스트, 딕셔너리 → DataFrame으로 변환

DataFrame 생성

- DataFrame = 컬럼명 有, 변환 시 컬럼명 지정(or 자동 할당)
- 판다스 DataFrame 객체 생성 인자 data = 리스트, 딕셔너리, 넘파이 ndarray 입력
- 생성 인자 columns = 컬럼명 리스트 입력

*DataFrame = 행과 열을 가지는 2차원 데이터, 따라서 2차원 이하 데이터만 변환 가능

d. DataFrame → 넘파이 ndarray, 리스트, 딕셔너리로 변환

ndarray로 변환: values

리스트로 변환: values로 얻은 ndarray에 tolist() 호출

딕셔너리로 변환: DataFrame 객체의 to_dict() 메소드 호출, 인자 'list' 입력 시 딕셔너리 값이 리스트형으로 반환

DataFrame의 컬럼 데이터 세트 생성과 수정

a. [] 연산자 사용하여 컬럼 데이터 세트 생성, 수정

→ DataFrame [] 내에 새로운 컬럼명 입력, 값 할당

b. DataFrame 데이터 삭제

→ drop() 메소드 사용

↳ 원형: DataFrame.drop(labels=None, axis = 0, index = None, columns = None, level = None, inplace = False, errors = 'raise')

→ axis 값: 특정 컬럼, 특정 행 드롭 / axis 0 로우 방향 축, axis 1 컬럼 방향

→ 원본 DataFrame 유지, 드롭된 DataFrame 새롭게 객체변수로 받을 때, inplace = False 설정

→ 원본 DataFrame에 드롭된 결과 적용 시 inplace = True 적용

→ 원본 DataFrame에서 도록된 DataFrame을 다시 원본 DataFrame 객체 변수로 할당 시, 원본 DataFrame에서 드롭된 결과를 적용할 경우와 동일

Index객체

- DataFrame, Series의 레코드를 고유하게 식별하는 객체
- DataFrame.index / Series.index 속성 사용
- reset_index() 인덱스가 연속된 int 숫자형 데이터가 아닐 경우, 다시 이를 연속 int 숫자형 데이터로 만들 때 사

데이터 선택 및 필터링

a. DataFrame의 [] 연산자

- [] 연산자: 칼럼명의 리스트 객체, 인덱스로 변환 가능한 표현
- DataFrame의 [] 연산자와 넘파이의 []나 Series의 [] 가 다름
- 슬라이싱 연산으로 데이터 추출 부적절

b. DataFrame iloc[] 연산자

- iloc[]: 위치 기반 인덱싱 방식 동작 / 행과 열 위치, 0을 출발점으로 하는 세로축, 가로축 좌표 정숫값으로 지정하는 방식
- loc[]: 명칭 기반 인덱싱 방식 동작 / 데이터 프레임의 인덱스 값으로 행 위치를, 칼럼의 명칭으로 열 위치를 지정
- -1 인덱싱: 가장 마지막 데이터 값

c. DataFrame loc[] 연산자

- 명칭 기반 데이터 추출
- 행 위치: DataFrame 인덱스 값 / 열 위치: 칼럼명 입력, loc[인덱스값, 칼럼명] 형식 데이터 추출

d. 불린 인덱싱

- and 조건 &, or 조건 |, Not 조건 ~

정렬, Aggregation 함수, GroupBy 적용

a. sort_values - DataFrame, Series 정렬

주요 입력 파라미터

→ by: 특정 칼럼 입력 시 해당 칼럼으로 정렬 수행

→ ascending = True 오름차순(default), = False 내림차순

→ inplace = False 설정(default) 시 sort_valuse() 호출 DataFrame 유지, 정렬 DataFrame 결과 반환

→ inplace = True 설정 시 호출 DataFrame 정렬 결과 그대로 적용

b. Aggregation 함수 적용

→ min(), max(), sum(), count()... etc

c. groupby() 적용

→ 입력 파라미터 by에 칼럼 입력 시, 대상 칼럼으로 groupby됨

→ 호출 시, DataFrameGroupBy라는 다른 형태의 DataFrame을 반환

결손 데이터 처리

→ 결손데이터 = Null인 경우 = NaN

→ NaN 여부 확인 API = isna()

→ NaN 값을 다른 값으로 대체 API = fillna()

a. isna() 로 데이터 결손 여부 확인

→ Trun, False로 여부 확인 가능

b. fillna()로 결손 데이터 대체

apply lambda 식으로 데이터 가공

→ 파이썬에서 함수형 프로그래밍 지원 목적